GPU-accelerating sparse Laplace-Beltrami eigensolvers for cortical surfaces

CuPy's LOBPCG is the most viable GPU eigensolver for your generalized problem, but the field's dominant pattern — CPU eigsh plus GPU downstream compute — remains the pragmatic choice. No existing GPU eigensolver fully replicates SciPy's ARPACK shift-invert for generalized eigenproblems at 150K–300K scale. CuPy `eigsh` lacks M-matrix and sigma support entirely; CuPy PyTorch's `lobpcg` supports generalized form but has documented correctness bugs; JAX has no sparse eigensolver at all. The bottleneck is unambiguously the eigensolver (**30–120s** on CPU for 300 eigenpairs), while Laplacian assembly takes ~100ms and spectral descriptors run in sub-milliseconds on GPU. A phased approach — starting with CPU eigsh plus GPU descriptors, graduating to CuPy LOBPCG — minimizes risk while capturing most speedup.

CuPy LOBPCG is the strongest GPU option, but eigsh has critical gaps

CuPy's `eigsh` (v13.6+) does NOT support generalized eigenvalue problems. The function signature omits the `M=`, `sigma=`, `Minv=`, and `mode=` parameters entirely — it solves only the standard problem `Ax = λx` CuPy CuPy using a Jacobi algorithm, not ARPACK. Shift-invert mode is absent, and `which` is limited to `'LM'`, `'LA'`, and `'SA'`. CuPy CuPy

CuPy's `lobpcg`, however, fully supports the generalized form via `B=`: CuPy

```python
import cupy as cp
import cupyx.scipy.sparse as csp
from cupyx.scipy.sparse.linalg import lobpcg

L_gpu = csp.csr_matrix(L_scipy.astype('float64'))
M_gpu = csp.csr_matrix(M_scipy.astype('float64'))

X0 = cp.random.randn(n, 300, dtype=cp.float64)
eigenvalues, eigenvectors = lobpcg(L_gpu, X0, B=M_gpu, largest=False,
                                   maxiter=200, tol=1e-6)
```

The `largest=False` flag targets smallest eigenvalues directly without shift-invert, CuPy which avoids the sparse factorization bottleneck that makes shift-invert impractical on GPU. LOBPCG's requirement that **n >> 5k** is easily satisfied (150K >> 1,500). Each iteration performs sparse matrix-block vector multiply (SpMM) — a GPU-friendly kernel — plus a **900×900** dense eigenproblem (for 3k = 900), which is trivial on GPU. The orthogonalization cost grows as **O(n·k²)**, making it the true bottleneck for k=300.

PyTorch's `torch.lobpcg` also supports generalized eigenproblems via `B=` and runs on CUDA sparse tensors:

```python
import torch
L_torch = torch.sparse_csr_tensor(
    torch.tensor(L_scipy.indptr, dtype=torch.int64),
    torch.tensor(L_scipy.indices, dtype=torch.int64),
    torch.tensor(L_scipy.data, dtype=torch.float64),
    size=L_scipy.shape).cuda()
M_torch = torch.sparse_csr_tensor(
    torch.tensor(M_scipy.indptr, dtype=torch.int64),
    torch.tensor(M_scipy.indices, dtype=torch.int64),
    torch.tensor(M_scipy.data, dtype=torch.float64),
    size=M_scipy.shape).cuda()

eigenvalues, eigenvectors = torch.lobpcg(
    A=L_torch, k=300, B=M_torch,
    X=torch.randn(n, 300, dtype=torch.float64, device='cuda'),
    largest=False, niter=200, tol=1e-6)
```

However, **torch.lobpcg has known correctness bugs** — GitHub issue `#101075` documents cases where it produces wrong eigenvalues for generalized problems, GitHub and benchmarks show it is often slower than SciPy's CPU LOBPCG. GitHub GitHub Use with caution and validate against SciPy results.

NVIDIA cuSOLVER's sparse eigensolver (`cusolverSpXcsreigvsi`) computes only a single eigenpair at a time via shift-inverse power iteration, and the entire cuSolverSp module is deprecated in favor of cuDSS. NVIDIA NVIDIA The dense generalized solver (`sygvd`) would require **180 GB** for a 150K×150K matrix. cuSOLVER is not viable for this problem.

SLEPc via slepc4py is the most robust option — production-grade Krylov-Schur and LOBPCG with full generalized eigenvalue support (Upv) and GPU-accelerated SpMV via PETSc's `aijcusparse` matrix type. (ResearchGate) The catch: shift-invert still runs the factorization on CPU because "direct solvers and preconditioners are not yet prepared to run on the GPU." (ResearchGate) Installation complexity (PETSc + MPI + CUDA configuration) is the main barrier.

JAX has no sparse eigensolver and won't get one soon

`jax.scipy.sparse.linalg.eigsh` **does not exist** as of JAX v0.9.2 (March 2026). The module contains only iterative linear solvers: `cg`, `gmres`, and `bicgstab`. (JAX Documentation +2) GitHub issue #3112 requesting sparse eigensolvers has been open since May 2020, labeled "contributions welcome" — the JAX team has explicitly deprioritized this. (GitHub)

The only JAX-native eigensolver is `jax.experimental.sparse.linalg.lobpcg_standard`, (JAX Documentation) which has two critical limitations for this problem: it **does not support generalized eigenproblems** (no M matrix) and it **only finds largest eigenvalues** (no `largest=False`). (JAX Documentation) Working around both by negating the operator and pre-transforming with $M^{-1/2}$ is possible but loses the numerical advantages of the generalized formulation.

`jax.experimental.sparse` itself is no longer actively developed. The documentation states: "experimental reference implementations, not recommended for performance-critical applications." (JAX Documentation) BCOO format supports JIT-compatible SpMV on GPU (JAX Documentation) (GitHub) (~16 MB for the sparse matrix), so a custom Lanczos or LOBPCG could theoretically be built in JAX — but reimplementing implicit restart, reorthogonalization, and convergence monitoring to match ARPACK's robustness is a major engineering effort with no existing library to build on.

The practical JAX path is **CPU eigsh +** `jax.device_put` **transfer**:

```python
import jax
import jax.numpy as jnp
jax.config.update("jax_enable_x64", True)

# CPU solve
eigenvalues, eigenvectors = scipy.sparse.linalg.eigsh(
    L, k=300, M=sp.diags(M_diag), sigma=0.0, which='LM')

# Transfer to JAX GPU: ~16ms for 360 MB on PCIe 4.0
eigvecs_gpu = jax.device_put(jnp.array(eigenvectors), jax.devices('gpu')[0])
eigvals_gpu = jax.device_put(jnp.array(eigenvalues), jax.devices('gpu')[0])
```

This is exactly what DiffusionNet and geometric-kernels do. Cross-framework transfer via DLPack (e.g., CuPy → JAX) is zero-copy if eigenpairs are already on GPU from a CuPy solver. APXML

Every geometry library computes eigenpairs on CPU

A comprehensive survey of specialized geometry libraries reveals a universal pattern: **no library provides GPU-accelerated LB eigenpair computation.**

robust-laplacian PyPI outputs only the sparse L and M matrices (as SciPy CSC) — no eigensolver at all, no GPU path. Its value is producing high-quality intrinsic Delaunay cotangent Laplacians. GitHub **libigl** provides `cotmatrix`, `massmatrix`, and `eigs` — all CPU-only via Eigen, no CUDA backend. Libigl **DiffusionNet** uses `scipy.sparse.linalg.eigsh` GitHub with `sigma=-0.01` shift-invert, caches results to `.npz` files, and converts to PyTorch tensors for the GPU-based neural network. This precompute-and-cache pattern is the de facto standard. **potpourri3d** provides geodesic distances and PDE-based mesh tools GitHub but no spectral computation at all. **geometric-kernels** uses robust-laplacian for matrix assembly Geometric-kernels and SciPy eigsh for eigensolves, with only kernel evaluation (post-eigenpairs) running on the selected GPU backend.

PyTorch3D is a notable partial exception — `pytorch3d.ops.cot_laplacian(verts, faces)` assembles the cotangent Laplacian **on GPU** as a sparse PyTorch tensor. Readthedocs But it provides no eigensolver, and its Laplacian is row-normalized (not the raw form needed for the generalized problem). Readthedocs **PyTorch Geometric** explicitly converts to SciPy sparse and calls CPU `eigsh` for Laplacian eigenvector positional encodings. PyTorch Geometric NVIDIA Kaolin offers only a uniform (not cotangent) Laplacian in dense format Readthedocs — impractical for 150K+ vertices.

Spectral descriptors map directly to GPU BLAS at sub-millisecond latency

Once eigenpairs live on GPU, the spectral feature matrix computation is a textbook dense BLAS pattern:

```python
# HKS: (N, T) = Φ² @ exp_weights, where exp_weights[k,j] = exp(-λ_k · t_j)
def compute_hks_gpu(eigenvalues, eigenvectors, time_scales):
    exp_weights = cp.exp(-eigenvalues[:, None] * time_scales[None, :])  # (K
    phi_sq = eigenvectors ** 2                                          # (N
    return phi_sq @ exp_weights                                         # (N
```

This is a **(150K × 300) × (300 × 42)** matrix multiply — a tall-skinny GEMM that requires ~**3.78 GFLOP**. On an A100 at 19.5 TFLOP/s FP64, theoretical compute time is ~**0.2 ms**. The operation is actually **memory-bandwidth-bound**: reading the 360 MB Φ^2 matrix at ~2 TB/s (A100 HBM) takes ~0.18 ms. Expected wall-clock time: **0.2–0.5 ms on GPU** versus ~7 ms on CPU (50 GB/s DDR4 bandwidth). Speedup is ~**10–30×**, though the absolute times are small either way. WKS and GPS follow the same pattern with different weight matrices.

The `spectral_feature_matrix` producing (N_vertices, 42) from (N_vertices, K) eigenvectors and (K,) eigenvalues is indeed a pure GEMM call, directly serviced by cuBLAS (`cublasGemm`) under the hood of CuPy's `@` operator or PyTorch's `torch.mm`.

MSN/SSN are too small for GPU; replace NetworkX with igraph

At **200 ROIs**, the correlation and cosine similarity computations produce 200×200 matrices from 200×10 feature profiles. This is ~5 orders of magnitude below the GPU benefit threshold. Benchmarks show CuPy `corrcoef` is **2–3× slower than NumPy** for matrices below ~100×1,000 due to kernel launch overhead (~10–100 μs per kernel). `np.corrcoef()` on a 200×10 matrix completes in **microseconds**. Keep MSN and SSN on CPU unless the data is already resident on GPU from a prior step, in which case avoid the transfer by using `cp.corrcoef()`.

For graph metrics, the 200-node scale makes GPU counterproductive. With **nx-cugraph** on a 34-node Karate Club graph, GPU took ~20 seconds versus 0.9 seconds on CPU. Medium The recommended replacement for NetworkX is **igraph** or **graph-tool**, both providing **10–100× speedup** over NetworkX on CPU with comprehensive algorithm coverage:

Metric	cuGraph (GPU)	igraph	graph-tool	NetworKit
Betweenness centrality	✅	✅	✅	✅
Clustering coefficient	Partial	✅	✅	✅
Global/local efficiency	❌	✅	✅	✅
Modularity (Louvain)	✅	✅	✅	✅
Rich club coefficient	❌	❌	✅	❌

cuGraph becomes essential only at vertex-level graphs (150K+ nodes), where betweenness centrality on CPU takes hours. ResearchGate For parcellation-scale analysis (200–1,000 nodes), igraph handles everything in under a second.

Memory budget and expected speedups for the full pipeline

For LOBPCG with k=300 on a 150K-vertex mesh:

Component	float64	float32
Sparse matrix (CSR)	12 MB	8 MB
LOBPCG workspace (6 block-vectors of n×k)	2.16 GB	1.08 GB
Output eigenpairs	360 MB	180 MB
Temporaries + Rayleigh-Ritz	~200 MB	~100 MB
Total GPU memory	**~2.7 GB**	**~1.4 GB**

This fits comfortably on any **8 GB+ GPU**. Using float32 for the eigensolver halves memory and typically doubles throughput (SpMM is memory-bandwidth-bound, and consumer GPUs have 1/32 or 1/64 the FP64 throughput of FP32). The low-frequency eigenpairs of the LB operator are well-conditioned enough for float32.

Benchmark data from Inductiva.AI (CuPy Jacobi eigsh on sparse tridiagonal matrices) shows GPU speedup scaling with matrix size: (Inductiva) **~7× at 100K, ~19× at 1M**. (inductiva) For the CorticalFields problem at 150K with LOBPCG (which is more GPU-friendly than Lanczos due to SpMM blocking), **expected speedup is 3–10×**, bringing eigensolver time from **30–120 seconds (CPU) to 5–30 seconds (GPU)**. The main bottleneck shifts from SpMM to orthogonalization at k=300, which grows as $O(n \cdot k^2)$.

CPU→GPU transfer of pre-computed eigenpairs is negligible: 360 MB at PCIe 4.0 (~22 GB/s practical) takes **~16 ms** (Intel) — less than 0.1% of the CPU eigensolver time. This confirms that GPU eigensolver adoption should be driven by eigensolver speedup, not transfer avoidance.

Normative model already benefits from CPU-then-transfer pattern

The SpectralMaternKernel evaluates `k(x,y) = (σ²/C) Σᵢ S(λᵢ) φᵢ(x) φᵢ(y)` where `S(λ) = (2ν/κ² + λ)^(-(ν+d/2))`. (NeurIPS) This is a dense matrix multiply `K = Φ · diag(S(λ)) · Φᵀ` — the same GPU-friendly GEMM pattern as spectral descriptors. Since GPyTorch training runs on GPU iteratively, the eigenpairs need to be on GPU once and are reused across many GP iterations. Transferring via `torch.from_numpy(eigvecs).to('cuda')` at **~16 ms** is negligible compared to typical GP training runs of minutes to hours. Computing eigenpairs on GPU avoids this transfer but adds complexity for marginal benefit.

Recommended backend architecture and phased implementation

The cleanest backend abstraction follows the **array-api-compat pattern** used by SciPy ≥1.11 (SciPy) and scikit-learn, with custom dispatch for sparse eigensolvers:

```python
# Graceful fallback
try:
    import cupy; HAS_CUPY = cupy.cuda.is_available()
except ImportError:
    HAS_CUPY = False

def eigsh_backend(L, M, k=300, backend="auto"):
    if backend == "auto":
        backend = "cupy" if HAS_CUPY else "scipy"

    if backend == "cupy":
        import cupyx.scipy.sparse as csp
        from cupyx.scipy.sparse.linalg import lobpcg
        L_gpu = csp.csr_matrix(L)
        M_gpu = csp.csr_matrix(M)
        X0 = cupy.random.randn(L.shape[0], k, dtype=cupy.float64)
        return lobpcg(L_gpu, X0, B=M_gpu, largest=False, tol=1e-6)
    else:
        return scipy.sparse.linalg.eigsh(L, k=k, M=M, sigma=0.0, which='LM')
```

Use **per-function dispatch** (not global backend) — this allows CPU assembly with GPU eigensolver, or CPU eigensolver with GPU descriptors, mixing freely based on what's available and beneficial.

Implementation phases:

- **Phase 1 (immediate, zero risk):** Keep SciPy eigsh on CPU. Transfer eigenpairs to GPU for spectral descriptors and GPyTorch. Replace NetworkX with igraph for graph metrics. This alone eliminates the two largest sources of unnecessary slowness.
- **Phase 2 (moderate effort, 3-10× eigensolver speedup):** Add CuPy LOBPCG as an optional backend with try/except fallback. Validate numerically against SciPy results on test meshes. Use float32 for the eigensolver to maximize GPU throughput.
- **Phase 3 (full GPU pipeline):** GPU cotangent Laplacian assembly (PyTorch3D-style). Direct CuPy→PyTorch handoff via DLPack for GPyTorch. Backend abstraction with array-api-compat for dense operations and custom dispatch for sparse.

The diagonal lumped mass matrix enables a useful escape hatch: transform to standard form via `M⁻¹/² L M⁻¹/² ψ = λ ψ` (trivial diagonal scaling), making even CuPy's limited `eigsh` usable — though LOBPCG with `B=` is cleaner and avoids potential numerical issues from the transformation.

Conclusion

The sparse eigensolver is the clear computational bottleneck at **30–120 seconds** versus milliseconds for everything else. **CuPy LOBPCG** is the most mature GPU option supporting the generalized form, CuPy with expected **3–10× speedup** arXiv and a **~2.7 GB** memory footprint that fits on commodity GPUs. JAX is a dead end for this problem — no sparse eigensolver exists or is planned. GitHub Every geometry processing library (DiffusionNet, geometric-kernels, robust-laplacian) uses the same CPU-eigsh-then-transfer pattern, ACM Digital Library Polytechnique validating it as the field standard. The spectral descriptor computation after eigenpairs is a pure GEMM completing in **<1 ms** on GPU. For network analysis at parcellation scale (200 nodes), replacing NetworkX with igraph delivers 10–100× speedup without GPU complexity, while cuGraph becomes essential only at vertex-level (150K+ node) graphs. RAPIDS The most impactful first step is not GPU eigensolvers but simply transferring CPU-computed eigenpairs to GPU and switching graph backends — capturing most of the achievable acceleration with minimal code changes.